

From a Question to a **Pattern Card**

A four-stage lifecycle for one demo in this lab — **01, 02, 03, 04**. It takes its shape from the *Development Playbook* and keeps only the four stages a demo really has. Every stage names its owner, its inputs, its deliverables, where AI helps, where it must not decide, and an exit test somebody can actually run.

THE CLAIM THIS GUIDE MAKES

A demo does two jobs, and they are not the same job. It **showcases a feature** so a person can see it work, and it **validates a technical slice** so a decision can be made on measured evidence. A demo that never says which job it is doing will do neither well.

HOW TO USE IT

This is a **working guide, not a gate**. Four stages, numbered **01** to **04**, and that is the whole list — a demo is never planned as a release, never operated and never supported, so the development playbook's **04–07** have no demo equivalent. Stage 04 feeds straight back into stage 01 of the next demo.

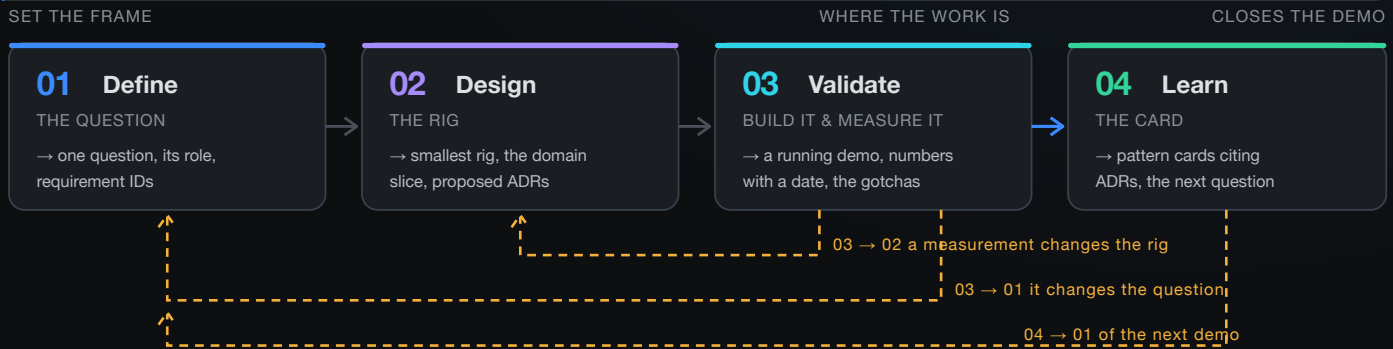
THE FOUR STAGES

- | | |
|---|--|
| ■ 01 Define · the question, and who is asking | ■ 03 Validate · build the slice and measure it |
| ■ 02 Design · the smallest rig that can answer it | ■ 04 Learn · the card, and the next question |

THE SHAPE OF THE WORK

The demo lifecycle

Four stages, borrowed numbers. The solid line is the order you first walk it. The dashed lines matter more: **evidence travelling backwards**. A demo drawn as a straight line is a demo that has never been corrected by a measurement.



A QUARTER WITH NO DASHED PATH USED IS A QUARTER WHERE NOTHING WAS MEASURED

01 · DEFINE

Done once per demo.
Redone when a measurement proves the question was the wrong one.

02 · DESIGN

The rig and the domain slice.
Small enough that stage 03 can start this week.

03 · VALIDATE

Build and measure are one stage. The code exists to produce the number.

04 · LEARN

The only stage that produces something an architect reads.
The demo is not done without it.

WHY A DEMO HAS FOUR STAGES AND NOT EIGHT

The *Development Playbook* has eight. Four of them have no demo equivalent. **04 Plan** and **06 Build** size and ship an increment of a product — a demo's build is subordinate to its measurement, so it lives inside **03 Validate**. **05 Enable** and **07 Release** make something supportable; a demo is torn down. So a demo's **04 Learn** is the development playbook's **08 Learn**, renumbered because there is nothing between them. **A demo is not a delivery** — four stages says so more plainly than a gap ever did.

DECLARE THIS AT STAGE 01

A demo does two jobs

Showcase and validation are **different demos** with different audiences, different deliverables and different exit tests. A demo may do both, but it must say so, because the choice is what stages 03 and 04 are graded against.

ROLE A

SHOWCASE A FEATURE

Audience	a person, often not you
Answers	“what does this feature do, and what is it for?”
Deliverables	<code>README.md</code> as the lab-shell intro, a runnable script or UI, a teardown
Exit test	somebody else runs it from the README and sees the feature , with no help from the author
Fails when	it only runs on the author's machine, in the author's shell history

ROLE B

VALIDATE A TECHNICAL SLICE

Audience	a decision, and the person who owns it
Answers	“where are the sharp edges, and what will they cost us?”
Deliverables	a reproducible experiment, measured findings, the gotchas, an accepted or revised ADR
Exit test	a number, with the date, the machine and the tool version , and the command that produced it
Fails when	the finding is a prediction, or the rig is gone and cannot be re-run

HOW THE ROLE CHANGES EACH STAGE

STAGE	IF THE ROLE IS SHOWCASE	IF THE ROLE IS VALIDATION
01	Name the feature and the reader. “An architect should understand X in ten minutes.”	Name the assumption that is both uncertain and expensive to change . Ignore the cheap ones.
02	Design for a stranger: one command up, one command down, sensible defaults.	Design for repeatability: the smallest rig that can still produce the number, and a way to break it on purpose.
03	The proof is a person following the README and seeing it work.	The proof is execution output . A prediction is not a result.
04	Cards describe what the feature is for and when to reach for it.	Cards describe the constraint , its cost, and the fallback. They cite the ADR.

BOTH IS ALLOWED, AND COMMON

Most demos here are **both**. Say so, and then meet **both** exit tests. A demo that claims both and ships only the measurements is a validation demo with an unpaid debt — record it as such rather than pretending.

THE FAILURE THIS RULE EXISTS TO STOP

A validation demo with no `README.md` and no script. The evidence is excellent and nobody but the author can reproduce or re-run it. Six months later the goal has to be reconstructed from a git commit message.

COMMON TRAP · Letting the role drift silently. A demo that started as a two-day validation grows a UI, then a second service, and is now an unowned product nobody validated anything with.

01 DEFINE

02 DESIGN

03 VALIDATE

04 LEARN

01

SET THE FRAME · ONCE, THEN ON CHANGE

Define the question

What are we trying to find out, and who is asking? The stage where a wish — “let's look at gateways” — becomes one sentence somebody can be wrong about.

ACCOUNTABLE OWNER

You, as the architect. AI may draft the wording. It does not pick the question, and it does not decide the role.

START WITH

A NATS feature reviewed from the **source docs**, not from memory and not from a blog. Plus whatever the last demo's stage 04 pushed back at you.

ACTIVITIES

- › **Write one primary question with a pass or fail answer.** “When a region goes dark, who can still accept a JetStream write?” is a question. “Explore gateways” is not.
- › **Declare the role** — **SHOWCASE**, **VALIDATION** or **BOTH**. This is the single most load-bearing line in the whole document. See page 03.
- › **Give every requirement an ID.** **D03-R1**, **D03-R2**. Without an ID, stage 04 has nothing to point back at and the loop is a wish.
- › Name the **logistics slice** the demo will speak in, so the answer is readable by somebody who knows the business and not NATS.

DELIVERABLES

`demos/<demo>/README.md` opening with the primary question, the declared role, and a ranked requirement list with IDs. The README is also the lab-shell intro, so write it for a stranger.

WHERE AI HELPS · AND WHERE IT MUST NOT

Reads the NATS source docs and drafts candidate questions and requirement wording. **It must not settle the question from its own knowledge.** Require a link or a quoted doc section for every claim about how NATS behaves.

DO NOT START UNTIL

There is something you are genuinely unsure of, or something a colleague genuinely cannot picture. A demo that cannot change a decision or teach a person is a hobby.

COMPLETION TEST

One sentence, on the page, that a measurement could prove wrong. The role is written down. Every requirement has an ID. Somebody other than you can read the README and say what the demo is for.

MAKE IT CONCRETE FOR THIS LAB

Write the question as the thing you would have to tell a stakeholder if the answer came back badly — a lost write, a WAN bill, a region that cannot take an order. That framing keeps the demo pointed at the platform instead of at the feature.

COMMON TRAP · Starting at stage 03 because the rig is fun to build. The goal then only exists in a commit message, and has to be archaeologically recovered later.

01 DEFINE

02 DESIGN

03 VALIDATE

04 LEARN

02

SET THE FRAME · ONCE, THEN ON CHANGE

Design the rig

What is the smallest thing that can still answer the question? Every part you add is a part that can explain a result away.

ACCOUNTABLE OWNER

You. AI drafts configs, scripts and diagrams; you own the scope and every proposed ADR.

START WITH

The primary question, the declared role, the requirement IDs, and the existing demos — most rigs here are a variation of one already built.

ACTIVITIES

- **Hold everything still except the one thing you are varying.** A demo that changes the link and the accounts and the subjects at once produces a result you cannot attribute.
- **Choose the smallest rig, and write down why it is enough.** Six hand-written config files on the host beat a Compose stack when the question is about quorum.
- **Design the break before you design the happy path.** How will you cut the WAN, kill the leader, revoke the credential? A validation demo is mostly failure injection.
- **Name the collisions.** Ports, context names, store directories, stream names and process name patterns must not be able to touch another demo or a live lab.
- **Write the proposed ADRs now**, stating the options and the pros and cons, with the choice left open. Stage 03 is what closes them.

DELIVERABLES

The demo's own `CLAUDE.md` sealing the folder's rules, a port and naming table in the `README.md`, the rig itself designed but not yet trusted, proposed ADRs under `ADR/lab/`, and a diagram of the topology.

WHERE AI HELPS · AND WHERE IT MUST NOT

Excellent at generating config variants, teardown scripts and the topology diagram. **It must not choose the scope.** Left alone it adds a service, a frontend and a Compose file to a question that needed four processes.

DO NOT START UNTIL

Stage 01 is on the page. Designing a rig for an unwritten question produces a rig that answers whichever question it turns out to be good at.

COMPLETION TEST

You can say, in one sentence, what result would **disprove** the design. Nothing in the rig can collide with another demo or with a running lab. The teardown is written before the setup is run.

MAKE IT CONCRETE FOR THIS LAB

Every scratch process, config file, port and `nats` context gets a **demo-specific prefix**, so a cleanup pattern can never match somebody else's live work. This rule exists because an unprefixed `kill` has already killed a live lab twice.

COMMON TRAP · Building the rig on the real thing. If the question needs a freeze, a kill or a delete, it needs a throwaway copy — never the lab somebody is still using.

01 DEFINE

02 DESIGN

03 VALIDATE

04 LEARN

03

WHERE THE WORK IS · THE ONLY STAGE THAT PRODUCES EVIDENCE

Validate — build the slice and measure it

Build it, then make it fail on purpose, and write down what came out. Build and measure are one stage here, not two. The code exists to produce the number; it is not the deliverable.

ACCOUNTABLE OWNER

You, together with whoever owns the decision the experiment is meant to settle.

START WITH

The designed rig, the proposed ADRs, and each experiment already written as a **question with a pass or fail answer** — written *before* the rig is run.

ACTIVITIES

- › **Build the rig, and get the happy path working first.** You cannot read a failure if you have never seen the success.
- › **Then break it deliberately.** Freeze a region, kill a leader, cut the link, revoke the credential. Record the exact error code and message, not a paraphrase.
- › **Run each measurement more than once.** A result that changes between runs is not a finding — either explain it or exclude it and say you excluded it.
- › **Record the date, the machine and the tool versions** next to every number. A number without them cannot be trusted a month later.
- › **Write the gotchas as you walk into them.** A trap you hit yourself is worth more than one you read about, and it is the part nobody else can reproduce for you.
- › **If the role includes showcase, a stranger must be able to run it.** Stage 03 is not complete on measurements alone.
- › **Tear the rig down and prove it.** Then decide which code is kept and which is deleted.

DELIVERABLES

A running demo, findings marked **measured** or **inferred** with a date, the error codes, the written gotchas, the accepted or revised ADRs, and a clean teardown.

WHERE AI HELPS · AND WHERE IT MUST NOT

Builds the rig, the failure-injection scripts and the counters, and summarises output. **Require real execution output for every claim.** A prediction is not a result. AI must not mark a row *measured* that it reasoned its way to, and it must review its own output for meaning, not just for style.

DO NOT START UNTIL

Each experiment is written as a pass-or-fail question, and the teardown exists. An experiment invented while the rig is already running will be graded against whatever it produced.

COMPLETION TEST

Every question from stage 01 has either **evidence** behind it, or a **written limitation with a workable fallback**. Unmeasured rows say *unmeasured*. The rig is down and the live labs are untouched. If the role includes showcase, somebody else ran it.

MAKE IT CONCRETE FOR THIS LAB

Never fake a WAN cut with `docker network disconnect` — it also drops the container's published host ports, so you measure your own tooling instead of the cut. Freeze the processes instead. Traps like this one **are** the deliverable.

COMMON TRAP · Letting the rig quietly become permanent. A throwaway that was never reviewed for error handling, secrets or support is now something you own.

01 DEFINE

02 DESIGN

03 VALIDATE

04 LEARN

04

CLOSES THE DEMO · AND OPENS THE NEXT ONE

Learn — the card, and the next question

What do we now know, and what does it change? The only stage that produces something an architect reads. **A demo is not finished until this exists.**

ACCOUNTABLE OWNER

You. The conclusion is a judgement about the platform, and a judgement needs a name on it.

START WITH

The findings from stage 03, the gotchas, the closed ADRs, and the requirement IDs from stage 01 so each finding can be traced to the thing that asked for it.

ACTIVITIES

- › **Close every ADR the demo was meant to settle** — the options, the pros and cons, the choice, and the evidence. The ADR holds the reasoning.
- › **Build the pattern cards deck** — the one artifact an architect or developer reads as a quick reference. Cards **cite** the ADRs; they do not restate the argument.
- › **Retract what turned out to be wrong.** Mark it retracted in place and say what replaced it. A silently deleted finding is how a wrong idea comes back.
- › **Write down the next question** and hand it to stage 01. If nothing new is worth asking, say that too — that is also a result.

DELIVERABLES

`demos/<demo>/docs/<demo>-pattern-cards.html` and its `.pdf`, closed ADRs under `ADR/lab/`, updated session memory, and the next demo's primary question.

WHERE AI HELPS · AND WHERE IT MUST NOT

Drafts the cards and the ADR prose from the recorded findings. **It must not invent a recommendation the evidence does not support**, and it must not upgrade an *inferred* row to *measured*. Every card traces to a finding or is labelled as judgement.

DO NOT START UNTIL

The findings are marked measured or inferred, and the rig is down. Cards written from a rig that is still moving will describe a state that no longer exists.

COMPLETION TEST

A reader who was not there can pick an approach, and say **why** and **what it costs**, from the deck alone. Every requirement ID from stage 01 is either answered or explicitly carried forward.

THE SPLIT OF RESPONSIBILITY

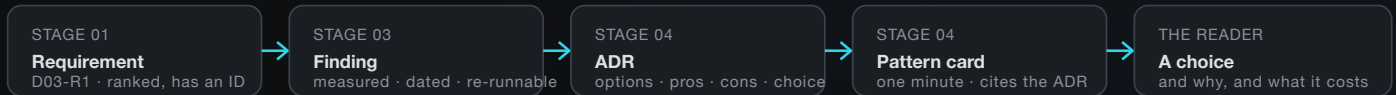
The ADR is the argument — the options, the trade-offs, the decision, the evidence, the requirement it answers. **The cards are the index** — one minute to read, and a pointer to the ADR when the reader wants the reasoning. One deck per demo. Do not invent a third artifact.

COMMON TRAP · Stopping at stage 03 because the answer is now obvious to you. It is obvious to you because you ran it. To everybody else the demo does not exist.

ONE ARTIFACT PER DEMO, AND WHAT FEEDS IT

The chain from question to choice

The deliverable a reader wants is a **choice with its pros and cons**, traced back to the requirement that asked for it. That is not one document. It is a short chain, and each link has one job.



A LINK YOU SKIP IS A CLAIM THE READER HAS TO TAKE ON TRUST

THE ADR IS THE ARGUMENT

Lives under `obsidian/V3-Platform/Architecture/ADR/lab/`. Holds the options that were on the table, the pros and cons of each, the decision, the evidence behind it, and the requirement ID it answers. This is where a **trade-off** belongs, because a trade-off needs room.

THE CARDS ARE THE INDEX

Lives at `demos/<demo>/docs/<demo>-pattern-cards.html` plus the exported `.pdf`. One deck per demo, read in a minute, with the constraint, its cost, the fallback, and a pointer to the ADR. **It cites; it does not restate.**

WHY NOT ONE SINGLE DOCUMENT

Because the two have different lifetimes. A **finding** is true of one version on one date and may later be **retracted**. A **decision** outlives the demo that produced it. Fusing them means either the reasoning is too long to be a quick reference, or the quick reference quietly becomes the only record of why. Keep the chain and keep the links short.

EXPORT

The deck is authored as HTML in the dark house style, then exported beside itself:

```
node demos/01-dictionary/diagrams/export-html-pdf.mjs <in.html> <out.pdf>
```

The HTML is the source of truth and the thing git can diff. The PDF is the thing you send. Same for this playbook.

COMMON TRAP · A deck of confident recommendations with no evidence behind any of them. If a card cannot name its finding or its ADR, it is an opinion wearing a card's clothes — label it as judgement or cut it.

WORKED EXAMPLE · AND WHAT TO CUT

Demo 03, scored against this guide

The demo that produced this playbook. It is a good demo with a real gap, which makes it the honest example. Scored 2026-09-17.

THE SCORE

STAGE	STATE	WHAT EXISTS, AND WHAT IS MISSING
01	missing	The question was excellent — “when a region goes dark, who can still accept a JetStream write?” — but it was never written on a page. It had to be recovered from a git commit message . No declared role, no requirement IDs.
02	partial	The rig is right: six hand-written configs, no Docker, one gateway, four accounts. Prefix and port discipline came later, after an unprefixed pkill hit a live lab. No README.md , no CLAUDE.md , no ADRs.
03	strong	Four topologies measured on the machine, not read from docs. Error codes recorded verbatim. Gotchas written down, including the measurement traps that produced wrong readings. A flaky result was excluded and said to be excluded . This stage is the model.
04	missing	No pattern cards deck. Demos 01, 02 and 04 all have one. The findings live in two HTML pages that only somebody who already knows the demo can navigate.

WHAT THE SCORE TELLS YOU

Strong evidence, no frame and no conclusion. So **the demo's role was never declared** — and because nobody declared it, it quietly became **VALIDATION**-only while everyone assumed it was **BOTH**. That is exactly the failure page 03 exists to stop.

THE FIX, IN ORDER

Write stage **01** and **02** retroactively as the **README.md** and **CLAUDE.md** — short, honest, dated. Then do stage **04** properly. Do **not** re-run stage 03; the evidence is good and re-measuring it would be theatre.

RIGHT-SIZING · WHAT TO CUT WHEN THE DEMO IS SMALL

- › **Never cut stage 01.** It is one paragraph and it is the cheapest thing in the book. Every cost in demo 03 traces back to skipping it.
- › **Never cut stage 04.** An unclosed demo is work nobody can use. If the deck would be three cards, write three cards.
- › **Stage 02 may collapse into a section of the README** for a small rig. It does not need its own document; it needs its decisions written down.
- › **Stage 03 may be one experiment.** One measured number beats four predicted ones.
- › **An ADR is only needed where a real choice was made.** A demo that confirmed the obvious produces cards and no ADR, and that is a complete demo.

APPLYING THIS GUIDE TO DEMOS ALREADY BUILT

Demos 01, 02 and 04 are built and carded. **Do not retro-fit them.** This is a working guide, not a gate, and back-filling paperwork for finished work teaches nobody anything. Apply it forward, and retro-fit only the demo you are closing right now.

COMMON TRAP · Treating the four stages as four documents. They are four questions. A small demo can answer all four inside one **README.md** and one card deck.